Techjam 2026: Task 3 Technical Report

Kevin Zhang, Isaac Ong, Hon Yi Hao, and Danny Chan

Group: Winner POV

***Abstract*—The disclosed test matrix for a fixed Transformer computation spans batch sizes, head dimensions and sequence lengths for which no single kernel performs best. We therefore submit a shape-aware dispatcher that selects among candidate kernel routes per input shape. Each route was first validated against the unmodified reference by comparing every output element against the stated tolerance, with any failing element disqualifying it, and only then timed using an interleaved measurement protocol. On an NVIDIA GeForce RTX 5080, the dispatcher passes 65/65 accuracy trials on test shapes 1–13 with zero failed elements, achieves a 3.611× geometric-mean speedup, and reduces peak allocation by up to 90.4%. It also makes test shape 14 executable in linear memory at 3.6 GiB, where it passes five trials against a validated FP32 oracle.**

1. Introduction

The task fixes the arithmetic of a Transformer layer and asks for GPU kernels that compute it faster while satisfying the executable per-element tolerance against a PyTorch reference. The reference, `torch_transformer_benchmark.py`, defines the model, inputs and checker; fourteen input shapes are disclosed in advance.

Table 1 shows why the design works better when it selects implementations by shape. We are able to categorise the disclosed test shapes into five different regimes:

1) **Launch-bound** (Cases 2, 3, 4, 12). A forward pass finishes in about a millisecond, so kernel launch and Python dispatch dominate.
2) **Memory-bound** (Cases 1, 5, 7, 9–11, 13). The $N \times N$ score tensor is read and written twelve times per layer to perform comparatively little matrix multiplication.
3) **Projection-bound** (Case 8). At $d_{\text{model}} = 1024$ and $N = 128$ the dense GEMMs dominate and attention is only 15.7% of the work.
4) **Capacity-bound** (Case 6). A batch of 10,000 makes the dense activation set exceed 10 GiB.
5) **Allocation-bound** (Case 14). At B=32, H=16, N=100,000 the score tensor holds 5.12×10^{12} values, or 20.48 TB (18.63 TiB) in FP32.

As such, an optimisation that wins in one regime may be neutral or harmful in another. For instance:

1) **QKV projection fusion** (merging the Q/K/V projection matmuls into one GEMM):
   - Case 3: $1.19\times$ speedup
   - Case 8: $1.003\times$ speedup (no benefit)
2) **Degree-2 polynomial feature map**:
   - Case 14: makes attention computation feasible
   - Case 6: makes attention $186\times$ *slower*
3) **FP16 internal execution behind an FP32 interface**:
   - Two-layer Case 14: passes
   - Four-layer shapes: fails

#	B	d_{model}	H	N	L	FFN	Class
1	64	128	4	128	4	128	memory-bound
2	1	128	4	128	4	128	launch-bound
3	4	128	4	128	4	128	launch-bound
4	16	128	4	128	4	128	launch-bound
5	128	128	4	128	4	128	memory-bound
6	10000	128	4	128	4	128	capacity-bound
7	64	32	4	128	4	32	memory-bound
8	64	1024	4	128	4	1024	projection-bound
9	64	128	1	128	4	128	memory-bound
10	64	128	2	128	4	128	memory-bound
11	64	128	16	128	4	128	memory-bound
12	64	128	4	32	4	128	launch-bound
13	64	128	4	1024	4	128	memory-bound
14	32	1024	16	100000	2	1024	allocation-bound

TABLE 1. THE FOURTEEN TEST SHAPES. CLASSIFICATION IS DERIVED FROM COST ANALYSIS AS OUTLINED IN §3.

From this analysis, we propose and submit a *decision-based* kernel dispatcher. We report:

1) A measurement methodology (§3) for a machine whose clocks move 2× under sustained load, including a paired interleaved schedule and a per-run noise floor.
2) A shape-aware dispatcher (§4) that routes on the disclosed tuple and runtime numerical contract.
3) Final benchmark results on a GeForce RTX 5080 (§5).
4) An account of how AI assistance was used (§3.4).

2. Problem and Correctness

2.1. Reference Computation

Each block applies pre-norm attention and a pre-norm GELU feed-forward network with residual additions, and the model applies a final LayerNorm. Here, attention is written in Listing 1:

```
scores = torch.matmul(q, k.transpose(-2, -1)) * self.
    scale
causal_mask = torch.ones((N, N), dtype=torch.bool,
                  device=x.device).triu(diagonal=1)
scores = scores.masked_fill(causal_mask, float("-inf"))
scores = scores.masked_fill(invalid_keys, float("-inf"))
probs = torch.softmax(scores.float(), dim=-1).to(dtype=x.
    dtype)
output = torch.matmul(probs, v)
```

Listing 1. The reference attention core, abridged from `BaselineSelfAttention.forward`.

2.2. Executable Criterion

An output element passes when

$$|c - r| \le 0.002 \quad \vee \quad |c - r| \le 0.02\,|r|, \tag{1}$$

where c is the candidate value and r the reference value.

2.3. Constraints

The reference rounds attention probabilities back to the model dtype before multiplying by V. Under a float16 reference, this makes free rewrites unsafe, as folding the $1/\sqrt{d_k}$ scale into Q causes both Cases 7 and 13 to fail. We therefore target float32 for the four-layer cases.

3. Analysis and Benchmarking Methodology

3.1. Analytic Cost Model

Each shape family was first characterised in order to better focus our optimisation strategies. For instance, for Case 13 (B=64, H=4, N=1024, d_h=32) the score tensor S holds 2.68×10^8 elements, or 1.00 GiB in FP32. Relative to the GPU's compute-to-bandwidth balance, the resulting arithmetic intensity places attention firmly in the memory-bound regime.

However, for Case 14, attention is about 95% of all FLOPs, roughly 0.655 PFLOP per causal layer. Hence, FLOP reduction is the biggest factor that can speed up computation.

3.2. Operator-Level Attribution

Analysis was confirmed using the PyTorch profiler with NVTX ranges at each stage. Figure 1 shows the result for Case 13 eager attention, as the actual attention matmul is the smallest cost, while the layout copies produced by `.contiguous()` in `_split_heads` are 7.1× larger.

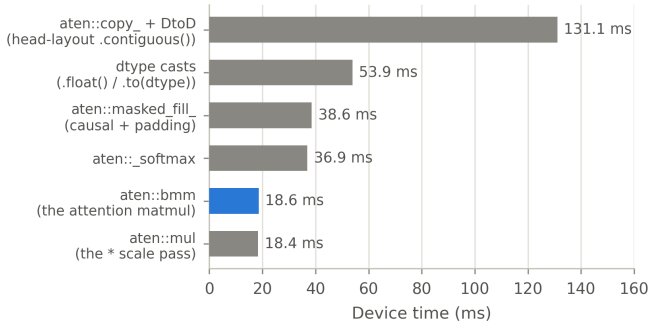

Figure 1. Where Case 13's eager attention time actually goes, by operator. The operator that performs the attention arithmetic is highlighted.

3.3. A/B Timing Measurement

As our development GPUs do not hold a stable clock, a schedule that measures one implementation before the other can report inconsistent speedups. From testing, identical code drifted 17.5% between sessions minutes apart, and four identical profiles spread 2.17×.

As such, we have implemented three mechanisms to address this, which can be found in `src/infra/timing.py`.

3.3.1. Settling. Iteration-count warmup is insufficient because the boost budget is spent in wall-clock time. Hence, each arm is instead held under continuous load for a fixed 10 seconds.

3.3.2. Interleaved paired schedule. Samples are emitted as alternating `AB` and `BA` pairs, which gives both labels the same mean position in the run and cancels monotonic drift.

3.3.3. Noise Floor from Within-Pair Disagreement. Because baseline and candidate are timed back to back, each pair sees the same clock and thermal state, so their ratio $\rho_i = b_i/c_i$ is unaffected by drift across the run. Any remaining variation between pairs reflects the run's own measurement noise. We quantify this noise floor as the half-width of a 95% confidence interval around the median of these ratios,

$$\phi \;=\; 1.96 \cdot \frac{1.2533 \cdot \mathrm{sd}(\rho)}{\sqrt{n}}, \tag{2}$$

where 1.2533 converts the standard error of the mean to that of the median. A measured speedup s is accepted only when

$|s - 1| > \phi$, i.e. only when it exceeds what the run's own noise could produce by chance. For launch-bound shapes, we batch several forward passes into each timed sample, sized automatically to target 50 ms per block to average out scheduling noise.

3.4. AI-Assisted Development

AI agents (Codex GPT 5.6 and Claude Opus 5) were used to perform research, repository and upstream-source inspection, task decomposition, profiler-trace interpretation, hypothesis generation, implementation, cross-stream conflict review, benchmark orchestration and documentation.

We only treat an AI suggestion as a hypothesis, and all optimisation changes were manually reviewed by each member before integrating them into the main branch. Here are some judgement calls we had to make in our implementation:

- An agent recommendation to pack Q/K/V universally was narrowed to Cases 2 and 3 after the end-to-end screen on Case 8 measured $1.003\times$.
- A blanket "FP16 fails" statement was found to be too general, as by testing, we found that it does not hold for FP16 internal compute behind an FP32 interface, which passes on Case 14.
- An attention-core speedup was correctly distinguished from a whole-model speedup after a microbenchmark and a full run disagreed by more than an order of magnitude.
- A numerical guard threshold initially proposed at 0.60, and later measured at 0.45, had to be lowered again to 0.40 after testing (§4.6.5).

4. Implementation

The submitted candidate is `DispatchingTransformer` in `src/dispatcher.py`. It subclasses the reference transformer, so it inherits the parameter names and block structure, and replaces only the attention module and the forward entry point.

4.1. Routing Key

Routing is decided on the tuple *and* the runtime numerical contract:

$$(B,\ N,\ d_{\text{model}},\ H,\ \text{FFN},\ \text{causal},\ L)$$

identifies the official case, and a separate runtime key carries the input shape, device type and index, dtype, device name and compute capability, mask presence, inference/grad state, float32 matmul precision and the TF32 flag. Anything else falls back to the reference arithmetic.

Compilation is lazy and cached per model instance and runtime key; the cache is invalidated by anything that could change correctness; and any compiled call that raises at runtime demotes permanently to the reference path.

4.2. Compiled Float32 SDPA

Cases 1–5 and 7–12 replace score materialisation with `scaled_dot_product_attention` [8] under `torch.compile(mode='reduce-overhead')` [9]; Cases 2 and 3 additionally use the packed-QKV route described below. Case 13 uses default compilation, which performed better for its larger compute- and memory-bound graph. This substitution delivers four of the levers identified in §3 at once:

1) the score tensor is never materialised [2], [1];
2) the softmax runs as a single online pass [3];
3) normalisation is deferred to the $[B, H, N, d_h]$ output;
4) causal block-skipping is already implemented inside the operator.

Compiler mode was chosen via analysis and measurement. For instance, for launch-bound shapes like Case 2, `reduce-overhead` measured $6.581\times$ against only $2.212\times$ for default mode, since `reduce-overhead` is specifically designed to cut this per-call overhead via CUDA graphs.

4.3. Layout and Mask Elimination

SDPA accepts strided inputs, so `.contiguous()` copies in `_split_heads` are avoidable. Hence, replacing them with transposed views adds a further 6–12% on top of SDPA and removes the largest item in Figure 1.

Additionally, we can remove the padding key mask for right-padded masks, as causal masking already writes $-\infty$ at every position the padding mask would, and the reference separately zeroes the invalid query rows. This condition is checked as a left-padded or interior-hole mask is not removable, so the dispatcher classifies the mask once per runtime key on the host and bakes the decision in before tracing.

4.4. Case 2 & 3: Packed QKV Projections

Cases 2 and 3 combine the three projection GEMMs into one. Both are launch-bound shapes, where runtime is dominated by per-kernel launch and dispatch overhead rather than by the GEMMs' own compute time; collapsing three launches into one removes overhead directly in the regime where overhead is the bottleneck. The packed weight is rebuilt outside timed execution, refreshed by a `load_state_dict` post-hook, and preserves the reference parameter names so strict weight loading still succeeds.

4.5. Case 6: Batch Streaming

Case 6's difficulty is being capacity-bound because a batch of 10,000 does not fit in memory at once. We handle this

by streaming the batch through a bounded float32 SDPA path in chunks. Chunk size is chosen as the largest power of two that fits a memory budget computed from live free memory, the reusable allocator cache, and a cap of 85% of total VRAM. If an allocation still fails, the chunk size is halved and the attempt is retried.

4.6. Case 14: Streaming, Mixed Precision and Guarded Linear Attention

Case 14 is the only shape that cannot execute from the reference arithmetic, as its dense FP32 score tensor is 20.48 TB (18.63 TiB), and its input and output together occupy about 24.4 GiB. Hence, the following sections elaborate on the three separate mechanisms that enable it to run.

4.6.1. Prefix streaming. The route processes one sample's valid prefix at a time and passes no attention mask. It therefore avoids retaining full-batch intermediate activations proportional to $B \cdot N \cdot d_{\text{model}}$, while the bounded working set for each streamed sample still scales with $N \cdot d_{\text{model}}$.

4.6.2. FP32 interface, FP16 compute. Parameters and the input/output interface remain FP32. Each streamed sample is cast to FP16 internally, and both Transformer blocks execute their attention, feed-forward and residual arithmetic in FP16 before the result is returned through the FP32-facing route.

4.6.3. Polynomial feature-map attention. At N=100,000 the path is compute-bound, so we focused on reducing FLOPs directly. We approximate $\exp(s)$ with the degree-2 polynomial that is L^2-optimal under the measured score distribution $s \sim \mathcal{N}(0, \sigma^2)$. The Gauss–Hermite projection gives

$$\exp(s) \;\approx\; e^{\sigma^2/2}\left[\left(1 - \tfrac{\sigma^2}{2}\right) + s + \tfrac{s^2}{2}\right], \qquad (3)$$

The $e^{\sigma^2/2}$ factor must be kept, since the block computed exactly along the diagonal uses unscaled exp and the two must remain on a consistent scale. Because Eq. 3 is a polynomial in $s = a \cdot b$, it can be rewritten as an explicit inner product. With $a = q\sqrt{\text{scale}}$ and $b = k\sqrt{\text{scale}}$, the feature map

$$\phi(a) = \left[\sqrt{c_0},\ \sqrt{c_1}\, a,\ \sqrt{c_2}\,\text{vec}(aa^\top)\right]$$

satisfies $\phi(a)\cdot\phi(b) = c_0 + c_1(a\cdot b) + c_2(a\cdot b)^2 \approx \exp(a \cdot b)$. Substituting this feature map turns causal attention into a chunked linear scan with a running state of size $O(d_f d_h) = O(d_h^3)$ and total work $O(N d_f d_h) = O(N d_h^3)$, rather than the $O(N^2 d_h)$ work of exact attention [5]. The construction is a deterministic relative of Performer's random features [6] and the second-order Taylor map of BASED [4].

The polynomial approximation is least accurate for early tokens, whose short causal prefixes provide fewer keys over which approximation errors can average out. We therefore compute the first 4,096 tokens exactly, which costs only 0.17% of the full quadratic work but cuts maximum error from 6.6×10^{-3} to 1.8×10^{-3}.

The polynomial route uses three fused Triton entry points—quadratic apply, quadratic update and the causal diagonal—that construct $aa^\top$ directly in registers [7]. This avoids materialising the $C \times d_h^2$ feature tiles in HBM and removes roughly 51 GB of cumulative feature traffic per sample-layer at the Case 14 shape. A microbenchmark of the two fused quadratic kernels against an SDPA-based implementation of the same linear-attention path shows an approximately 1.5× speedup. The exact-prefix correction itself uses SDPA.

4.6.4. Where Crossover Favours Linear Attention. The method trades $O(N^2 d_h)$ for $O(N d_h^3)$, so it wins when

$$N \;>\; 2d_h^2. \qquad (4)$$

Figure 2 places all fourteen shapes against the boundary of Eq. 4, and only Case 14 is on the profitable side.

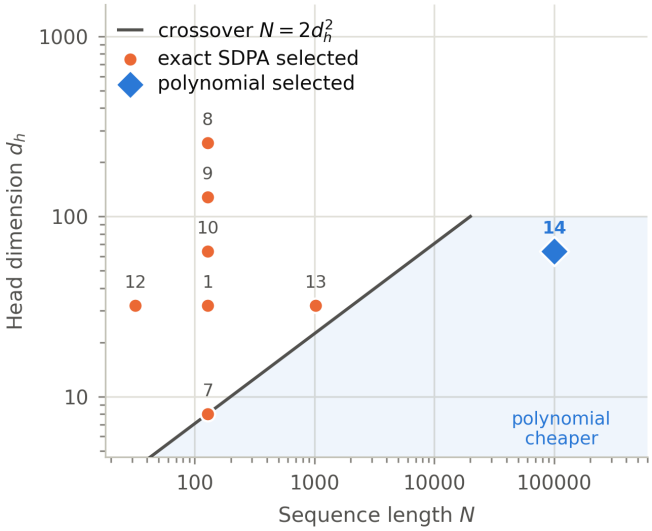

Figure 2. Linear attention is cheaper than exact causal attention only above $N = 2d_h^2$. Only Case 14 sits far inside the shaded region.

4.6.5. The runtime guard. The approximation's accuracy depends on σ, which is a property of the benchmark's random initialisation. Under trained weights, scores are far larger and a degree-2 fit would be poor. To prevent this, the route is gated at runtime: σ is estimated from a 512-row sample, and the polynomial path runs only when $\sigma \le 0.40$. Otherwise, the route forces exact Flash SDPA.

The ceiling was set by the sweep in Figure 3, obtained by scaling Q/K/V projection weights of both reference and candidate and recording where the criterion first fails. Additionally, the failure count is not monotonic near the boundary. Therefore, we placed σ (0.40) below the largest clean pass at 0.4188 and 20% above the operating point.

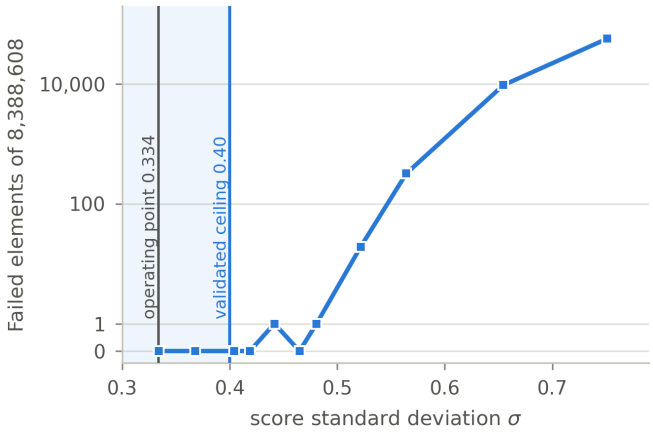

Figure 3. Calibration of the polynomial route's runtime guard, at N=8192. The shaded band is the admitted range.

Finally, we believe that σ is conservative for the target shape, because it was calibrated at N=8192 while attention contributes less to the residual stream as N grows.

5. Results

5.1. Environment

All results are obtained from the single machine in Table 2. The benchmark sets float32 matmul precision to `high`, enables TF32, uses input scale 1.0 with zero padding, and executes on CUDA 13.0.

Component	Final benchmark machine
CPU	AMD Ryzen 7 9800X3D, 8 cores / 16 threads
CPU cache	384 KiB L1d, 256 KiB L1i, 8 MiB L2, 96 MiB L3
GPU	NVIDIA GeForce RTX 5080, compute capability 12.0
GPU memory	16,303 MiB
GPU limits	360 W; max queried SM clock 3,090 MHz; memory clock 15,001 MHz
Driver / CUDA	NVIDIA 616.56 / CUDA 13.0
cuDNN	9.2.0
PyTorch / Triton	2.13.0+cu130 / 3.7.1
Python	3.12.3
OS	WSL2, Linux 6.6.114.1, x86-64, glibc 2.39
Disk	1,007 GiB filesystem; 616 GiB free before the run

TABLE 2. FINAL BENCHMARK ENVIRONMENT.

We report accuracy averaged over five seeds (1234–1238). Timing follows the protocol in §3.3: we interleave paired CUDA-event measurements between arms, let each arm settle for 10 seconds before recording anything, and collect 100 paired samples per comparison. Block size is chosen automatically so each timed block runs for roughly 50 ms. Memory is measured after route compilation finishes and only once replays are warm.

5.2. Cases 1–13

Table 3 and Figure 4 give directly comparable results. Every case passes 5/5 accuracy trials with zero failed elements, and every improvement clears its own noise floor. The geometric-mean speedup is **3.611**×, ranging from 1.116× on projection-bound Case 8 to 8.577× on long-sequence Case 13.

Case	Base ms	Cand. ms	Speedup	Noise	Max abs
1	1.9350	0.6975	2.774×	25.30%	0.001054
2	1.1953	0.1609	7.428×	62.49%	0.000819
3	0.9788	0.1643	5.959×	34.53%	0.000908
4	1.1567	0.2678	4.319×	17.64%	0.000908
5	3.0159	1.2513	2.410×	8.85%	0.001181
6	504.1028	210.5723	2.394×	0.52%	0.001347
7	1.5418	0.4060	3.797×	33.12%	0.001316
8	17.3913	15.5889	1.116×	0.82%	0.001218
9	1.5037	0.6881	2.185×	11.03%	0.001078
10	1.8052	0.6464	2.793×	20.44%	0.001089
11	7.6778	1.3724	5.595×	7.27%	0.001063
12	1.0901	0.2335	4.668×	41.84%	0.001128
13	116.7147	13.6078	8.577×	9.12%	0.001123
Geometric mean			**3.611**×		

TABLE 3. RTX 5080 RESULTS AT COMMIT `775c820`. EACH ROW PASSES WITH ZERO FAILED ELEMENTS AND SIGNIFICANT SPEEDUP OVER NOISE PER EQ. 2.

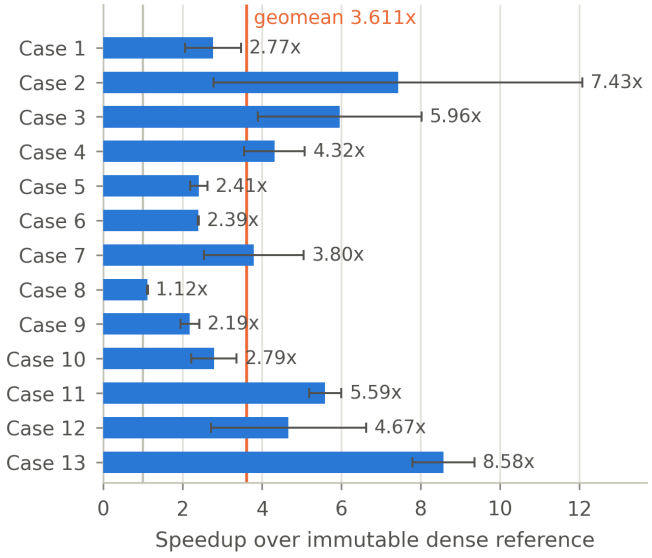

Figure 4. Per-case speedup over the provided reference, with run-specific 95% noise floor of Eq. 2 as error bars.

5.3. Latency and Memory

Figure 5 shows median latency and peak allocated memory side by side across all 13 cases. On the memory side, there is a clear reduction in peak allocated memory, particularly for

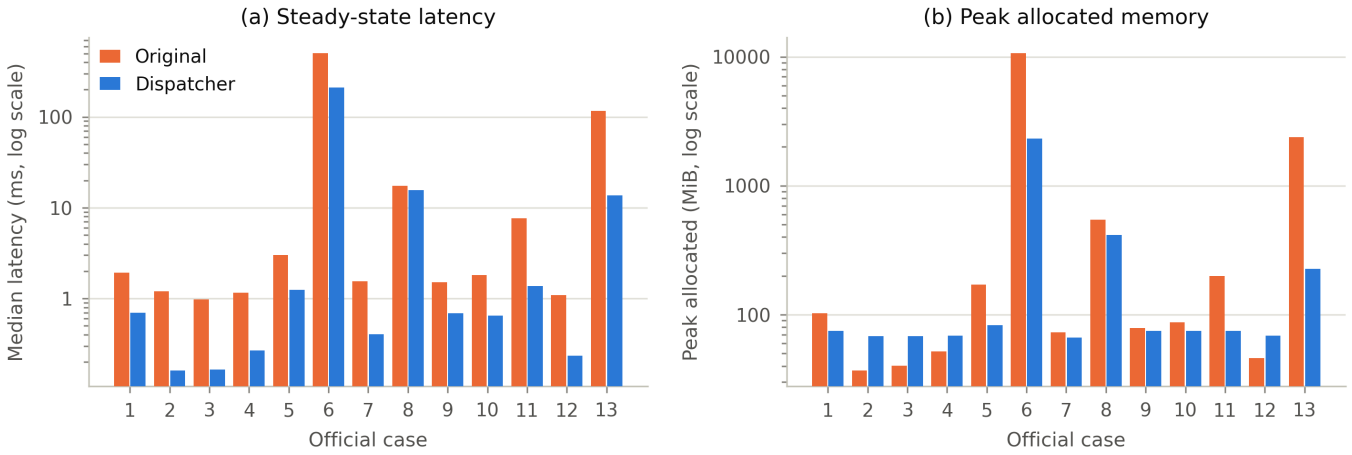

Figure 5. Cases 1–13 on the RTX 5080, both axes on log scales. (a) Median steady-state latency. (b) Peak allocated memory.

the memory-bound Case 6, where peak allocation dropped from 10,672.719 MiB to 2,312.512 MiB, a -78.3% reduction. While there are four cases (2, 3, 4, and 12) where the candidate's peak allocation is higher than the baseline's, all four are sub-70 MiB; thus the number is likely dominated by resident Inductor and CUDA Graph caches.

5.4. Case 14

We are unable to run the baseline for Case 14, as the original model cannot execute at the target shape, and separately, the reference accuracy checker exhausts memory at this output size because it retains full input, reference, and candidate tensors and then creates FP32 copies and boolean temporaries on top of them. We therefore report Case 14 separately in this section rather than folding it into the main results.

To still obtain a meaningful accuracy measurement, we designed a purpose-built harness that does the following:

1) Validates a linear-memory FP32 oracle against the immutable dense model at B=1, N=4096, obtaining 0/4,194,304 failures at $\max_{\text{abs}} = 0.00060248$.
2) Generates one deterministic FP32 sample at a time at the full target shape (B=32, N=100,000, d_{model}=1024).
3) Passes that identical sample to both the oracle and the FP32-facing candidate.
4) Reduces the comparison in bounded token chunks, discarding each sample's outputs immediately after comparison.

Table 4 gives the result.

Quantity	Value
Accuracy	PASS 5/5 trials, 0 failures across 16,384,000,000 elements
Max absolute error	0.010294 (passes via the 2% relative arm of Eq. 1)
Mean absolute error	0.000374
Oracle time	601.975 s
Candidate time	67.095 s
Diagnostic ratio	8.972×
Peak allocation	3,643.988 MiB
Measured σ per layer	0.3340, 0.3345 (guard ceiling 0.40)

TABLE 4. CASE 14 AGAINST THE VALIDATED STREAMED FP32 ORACLE (A LINEAR-MEMORY PROXY).

6. Discussion

The largest gains are on memory-bound attention (Case 13 at 8.577×, Case 11 at 5.595×), where removing the score tensor from memory removes most of the runtime. The next largest are on launch-bound shapes (Case 2 at 7.428×), where CUDA Graph replay removes per-call dispatch that was most of the measured time. The smallest is on projection-bound Case 8 at 1.116×.

Reproduction

The repository is at https://github.com/IceyFoxes/techjam. Figures in this report are regenerated from the preserved benchmark JSON by `docs/make_figures.py`. Any directly comparable case runs as shown in Listing 2, substituting `--case` 1 to 14.

```
.venv/bin/python -m src.benchmark \
  --candidate src.dispatcher --case 6 \
  --device cuda --dtype float32 \
  --accuracy-trials 5 --seed 1234 \
  --warmup 20 --repeats 100 --benchmark-rounds 3 \
  --timing paired --settle-seconds 10 \
  --output research/benchmarks/DATE-GPU-COMMIT/result.json
```

Listing 2. Reproducing one official case.

References

[1] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2307.08691

[2] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," arXiv:2112.05682, 2021. [Online]. Available: https://arxiv.org/abs/2112.05682

[3] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv:1805.02867, 2018. [Online]. Available: https://arxiv.org/abs/1805.02867

[4] S. Arora, S. Eyuboglu, M. Zhang, et al., "Simple linear attention language models balance the recall-throughput tradeoff," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.18668

[5] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020. [Online]. Available: https://proceedings.mlr.press/v119/katharopoulos20a.html

[6] K. Choromanski, V. Likhosherstov, D. Dohan, et al., "Rethinking attention with Performers," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021. [Online]. Available: https://arxiv.org/abs/2009.14794

[7] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19. [Online]. Available: https://doi.org/10.1145/3315508.3329973

[8] PyTorch, "torch.nn.functional.scaled_dot_product_attention," PyTorch documentation. Accessed 29 August 2026. [Online]. Available: https://docs.pytorch.org/docs/main/generated/torch.nn.functional.scaled_dot_product_attention.html

[9] J. Ansel, E. Yang, H. He, et al., "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. 29th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024. [Online]. Available: https://doi.org/10.1145/3620665.3640366